SYNAPSE

AI-Powered Technology Intelligence & Automation Platform

Complete Project Documentation Suite

Document Type	Master Documentation Index
Version	1.0.0
Date	March 2026
Architecture	FAANG-Style Production System
Classification	Confidential – Internal Use

Platform Vision

"An AI-powered technology intelligence and automation system that continuously collects information from the internet, analyzes it using AI, and allows users to interact with that knowledge and automate tasks."

Core Technologies

`Django` • `FastAPI` • `Next.js` • `PostgreSQL` • `Redis`

`LangChain` • `OpenAI` • `spaCy` • `Celery` • `Docker`

`Scrapy` • `pgvector` • `TailwindCSS` • `GitHub Actions`

SYNAPSE Development Team • 2026

Documentation Suite Overview

This master document serves as the central index for the complete SYNAPSE project documentation suite. SYNAPSE is a **FAANG-style, production-grade, AI-powered technology intelligence and automation platform** that combines data engineering, machine learning, natural language processing, agentic AI, and full-stack development into a single cohesive product.

What is SYNAPSE?

SYNAPSE is best described as an intelligent combination of:

- **Perplexity AI** — semantic search over curated technical knowledge
- **Notion** — personal knowledge management and document studio
- **Zapier** — no-code automation workflows
- **GitHub** — repository intelligence and trend monitoring

But specialized exclusively for **technology, AI, software engineering, and research**.

System Layers at a Glance

#	Layer	Description
1	Internet Sources	Hacker News, arXiv, GitHub, YouTube, Tech Blogs
2	Data Collection	Scrapy, BeautifulSoup, Playwright, API Workers
3	Message Queue	Redis + Celery (async task processing)
4	Processing Pipeline	Cleaning, deduplication, NLP preprocessing
5	Knowledge Storage	PostgreSQL, pgvector, Redis Cache, S3
6	AI/Intelligence Layer	NLP Engine, ML Recommendations, Semantic Search
7	Agentic AI Layer	LangChain agents, tool execution, file generation
8	API Layer	Django REST + FastAPI + GraphQL
9	Frontend Layer	Next.js, React, TailwindCSS, Framer Motion
10	User Interaction	Dashboard, AI Chat, Automation Control Panel

Documents in This Suite

Doc #	Document	Contents	Lines
01	Software Requirements Spec	Functional & non-functional requirements, FR-001–FR-050+	1,019
02	Architecture & Design	System layers, component design, data flow, patterns	1,893
03	Database Schema	All 17 tables, indexes, vector DB, Redis patterns	1,253
04	API Specification	All REST endpoints, request/response schemas, auth	2,079
05	Project Roadmap	22-week phased plan, Gantt chart, risk register	1,646
06	Implementation Guide	Step-by-step code guide, snippets, all components	2,028
07	UI/UX Design	Design system, component library, 10 page specs	1,103
08	DevOps & Deployment	Docker, CI/CD, AWS, Nginx, monitoring, security	2,585
	Total		**13,606**

How to Use This Documentation

Starting development?
Begin with **Doc 01 (SRS)** to understand requirements, then **Doc 02 (Architecture)** for system design.

Setting up the database?
Refer to **Doc 03 (Database Schema)** for all table definitions, indexes, and vector DB configuration.

Building APIs?
Use **Doc 04 (API Specification)** as your API contract.

Planning sprints?
Use **Doc 05 (Roadmap)** for the 22-week phased plan.

Writing code?
Doc 06 (Implementation Guide) has step-by-step instructions with actual code snippets for every component.

Designing the UI?
Doc 07 (UI/UX Design) contains the full design system and page-by-page specifications.

Deploying to production?
Follow **Doc 08 (DevOps)** for Docker, CI/CD, AWS infrastructure, monitoring, and security hardening.

Securing the platform?
Doc 09 (Security) covers OWASP Top 10, threat modelling, GDPR compliance, penetration testing checklist, and incident response.

Setting up testing?
Doc 10 (Testing Strategy) covers unit, integration, E2E, load, and AI/ML testing with full code examples.

Planning the business?
Doc 11 (Business Plan) has market analysis, competitor matrix, pricing tiers, financial projections, and go-to-market strategy.

Contents

Documentation Suite Overview **1**

1 Platform Architecture Summary **5**
- 1.1 High-Level Architecture . . . 5
- 1.2 Technology Stack Summary . . . 6
- 1.3 Key Features . . . 6
  - 1.3.1 Data Intelligence . . . 6
  - 1.3.2 AI Capabilities . . . 7
  - 1.3.3 Automation . . . 7
  - 1.3.4 Document Generation . . . 7
- 1.4 Frontend Pages . . . 7

2 Development Phases Summary **8**
- 2.1 22-Week Roadmap Overview . . . 8
- 2.2 Skill Requirements . . . 9

3 Database Quick Reference **10**
- 3.1 Core Tables . . . 10
- 3.2 API Endpoint Groups . . . 11

4 Project Vision & Success Metrics **12**
- 4.1 Vision Statement . . . 12
- 4.2 Success Metrics (KPIs) . . . 12
- 4.3 Comparable Products . . . 12
- 4.4 Conclusion . . . 13

Chapter 1

Platform Architecture Summary

1.1 High-Level Architecture

SYNAPSE follows a layered microservices-inspired architecture designed for horizontal scalability, AI-first processing, and production reliability. The data flow moves top-to-bottom through distinct functional layers:

```
INTERNET SOURCES
[Hacker News] [arXiv] [GitHub] [YouTube] [Tech Blogs]
      |
      v
DATA COLLECTION LAYER
[Scrapy Spiders] [API Collectors] [Playwright] [Celery Beat]
      |
      v
MESSAGE / TASK QUEUE
[Redis] [Celery Workers] [Kafka (optional)]
      |
      v
DATA PROCESSING PIPELINE
[HTML Cleaning] [Deduplication] [Metadata Extraction]
[spaCy NLP] [Topic Classification] [Embedding Generation]
      |
      v
KNOWLEDGE STORAGE LAYER
[PostgreSQL] [pgvector/Pinecone] [Redis Cache] [AWS S3]
      |
      v
AI / INTELLIGENCE LAYER
[NLP Engine] [Recommendation Engine] [Semantic Search]
      |
      v
AGENTIC AI LAYER
[LangChain Agents] [Tool Execution] [Document Generation]
      |
      v
```

```
API LAYER
[Django REST] [FastAPI] [GraphQL] [JWT Auth] [Rate Limiting]
      |
      v
FRONTEND LAYER
[Next.js] [React] [TailwindCSS] [Framer Motion] [Chart.js]
      |
      v
USER INTERACTION
[Dashboard] [AI Chat] [Automation Panel] [Document Studio]
```

1.2 Technology Stack Summary

Category	Technology	Purpose
Backend Framework	Django 4.2	Main application server, ORM, admin
AI Microservice	FastAPI 0.104	High-performance AI endpoints
Frontend	Next.js 14 + React 18	SSR dashboard and UI
Styling	TailwindCSS 3.4	Utility-first CSS framework
Animations	Framer Motion	Page transitions, micro-interactions
Charts	Recharts / Chart.js	Data visualizations
Primary DB	PostgreSQL 15	Structured data storage
Vector DB	pgvector / Pinecone	Embedding storage, semantic search
Cache	Redis 7	Session cache, task queue broker
Object Storage	AWS S3 / Google Drive	Files, documents, media
Task Queue	Celery 5.3 + Redis	Async background jobs
Web Scraping	Scrapy + Playwright	Data collection automation
AI Framework	LangChain	RAG pipelines, AI agents
LLM	OpenAI API (GPT-4)	Language understanding & generation
NLP Library	spaCy	Text processing, NER, classification
ML Library	HuggingFace Transformers	Summarization, embeddings
Containerization	Docker + Compose	Service packaging
CI/CD	GitHub Actions	Automated testing & deployment
Cloud	AWS (EC2, RDS, S3)	Production hosting
Monitoring	Prometheus + Grafana	Metrics and dashboards
Error Tracking	Sentry	Exception monitoring

1.3 Key Features

1.3.1 Data Intelligence

- Automated scraping every 30 minutes (news), 2 hours (GitHub), 6 hours (arXiv)
- NLP pipeline: summarization, topic classification, keyword extraction, sentiment analysis
- Technology trend radar tracking mention frequency across all sources

- Personalized recommendation engine based on user interests and reading history

1.3.2 AI Capabilities

- **Semantic Search**: Vector embeddings (OpenAI `text-embedding-ada-002` or `sentence-transformers`) stored in pgvector; natural language queries
- **AI Chat Assistant**: RAG pipeline with LangChain, retrieves from knowledge base, generates grounded responses with source citations
- **Autonomous Agents**: LangChain ReAct agents with tools for document generation, code scaffolding, web search, and cloud uploads

1.3.3 Automation

- User-configurable workflows with cron scheduling (Celery Beat)
- Actions: collect news, summarize, generate PDF/PPT/Word, send email, upload to Drive/S3
- Workflow run history, status tracking, in-app + email notifications

1.3.4 Document Generation

- AI-generated PDFs (ReportLab), PowerPoint presentations (python-pptx), Word documents (python-docx), Markdown reports
- Code project scaffolding (Django, FastAPI, React templates)
- Files auto-saved to Google Drive or AWS S3

1.4 Frontend Pages

Page	Description
Main Dashboard	Stats overview, trend radar, AI news feed, GitHub widget
Tech Intelligence Feed	AI-curated articles with summaries, filters, infinite scroll
Research Explorer	arXiv papers with AI summaries, difficulty levels, search
GitHub Radar	Trending repositories, language filters, star growth charts
Learning Hub	Educational videos, collections, reading lists
AI Chat Assistant	RAG-powered conversational AI with source citations
Automation Center	Workflow builder, active agents panel, run history
Document Studio	AI document generation, file library, downloads
Knowledge Library	Personal bookmarks, collections, notes, tag filters
Profile & Settings	User preferences, interests, notifications, API keys

Chapter 2

Development Phases Summary

2.1 22-Week Roadmap Overview

Phase	Weeks	Deliverable	Milestone
0	Pre-dev	Architecture, schema, API contracts	Documents ready
1	1–4	Core platform with live data scraping	Working dashboard
2	5–8	NLP pipeline, semantic search, recommendations	AI-powered platform
3	9–10	LangChain RAG chat assistant	AI assistant live
4	11–12	Automation workflows, notifications	Automation running
5	13–16	Agentic AI, document generation	Agents operational
6	17–18	Google Drive + AWS S3 integration	Cloud storage live
7	19–20	Premium UI/UX, animations, dark mode	Startup-quality UI
8	21–22	Docker, CI/CD, AWS deployment, monitoring	Production launch

2.2 Skill Requirements

Skill Area	Level Required	Primary Technologies
Web Scraping	Advanced	Scrapy, Playwright, BeautifulSoup
Backend Development	Advanced	Django, FastAPI, REST APIs
Frontend Development	Advanced	Next.js, React, TypeScript
Database Design	Advanced	PostgreSQL, Redis, pgvector
NLP / AI	Advanced	spaCy, HuggingFace, LangChain
Machine Learning	Intermediate	Recommendation systems, embeddings
AI Agents	Advanced	LangChain ReAct, tool use
DevOps	Intermediate	Docker, CI/CD, AWS
Cloud Integration	Intermediate	S3, Google Drive API
System Architecture	Advanced	Distributed systems, queues

Chapter 3

Database Quick Reference

3.1 Core Tables

Table	Purpose
`users`	User accounts, roles, preferences
`sources`	Scraping sources (news, GitHub, arXiv, YouTube)
`articles`	Scraped articles with NLP metadata and embeddings
`repositories`	GitHub repositories with star/fork counts
`research_papers`	arXiv papers with summaries and difficulty levels
`videos`	YouTube videos with transcripts
`user_bookmarks`	User-saved content across all types
`collections`	Named knowledge collections
`collection_items`	Items within collections
`automation_workflows`	User-defined scheduled workflows
`workflow_runs`	Execution history per workflow
`generated_documents`	AI-generated files (PDF, PPT, Word)
`agent_tasks`	Autonomous agent task queue and results
`technology_trends`	Daily technology mention tracking
`notifications`	In-app and email notification records
`user_activity`	User interaction logs for recommendations

3.2 API Endpoint Groups

Group	Prefix	Key Endpoints
Authentication	`/api/v1/auth/`	register, login, refresh, me
Articles	`/api/v1/articles/`	list, detail, trending, search
Semantic Search	`/api/v1/search/`	semantic vector search
Research Papers	`/api/v1/papers/`	list, detail, trending
Repositories	`/api/v1/repos/`	list, detail, trending
Videos	`/api/v1/videos/`	list, detail, transcript
AI Assistant	`/api/v1/ai/`	chat, summarize, explain
Agent Tasks	`/api/v1/agents/`	create task, status, tools list
Documents	`/api/v1/documents/`	generate, download, list
Workflows	`/api/v1/workflows/`	CRUD, trigger, run history
Trends	`/api/v1/trends/`	radar, timeline, categories
Collections	`/api/v1/collections/`	CRUD, items management
Notifications	`/api/v1/notifications/`	list, mark read
Admin	`/api/v1/admin/`	stats, scraper control, users

Chapter 4

Project Vision & Success Metrics

4.1 Vision Statement

SYNAPSE will become a **personal AI technology intelligence system** that:

- Monitors the tech world 24/7 automatically
- Analyzes information with state-of-the-art AI
- Answers complex technical questions with cited sources
- Automates repetitive research and reporting tasks
- Generates professional knowledge artifacts on demand
- Assists in development project scaffolding

4.2 Success Metrics (KPIs)

Category	**Metric**	**Target**
Performance	Page load time	< 2 seconds
Performance	API response time (p95)	< 500 ms
Performance	Semantic search latency	< 1 second
Reliability	System uptime	$\geq 99.5\%$
Reliability	Data freshness (news)	Every 30 minutes
Scalability	Concurrent users	$\geq 1,000$
Data	Articles in knowledge base	$> 10,000$
Data	Research papers indexed	$> 5,000$
AI Quality	Chat response relevance	$\geq 85\%$ user satisfaction
AI Quality	Summary accuracy	ROUGE-L ≥ 0.4
Security	Vulnerability response	< 24 hours

4.3 Comparable Products

SYNAPSE draws inspiration from and competes with:

- **Perplexity AI** — AI-powered search with citations
- **Notion AI** — Knowledge management with AI assistance
- **Zapier** — No-code automation workflows
- **GitHub** — Developer intelligence and trend tracking
- **Feedly** — RSS-based content aggregation
- **Semantic Scholar** — AI-powered research paper discovery

What makes SYNAPSE unique is its **vertical specialization** in technology and AI, combined with **autonomous agentic capabilities** for document generation and task automation – a combination not offered by any single existing product.

4.4 Conclusion

This documentation suite represents a **complete pre-development blueprint** for the SYNAPSE platform. With 13,606 lines of professional technical documentation across 8 specialized documents, every aspect of the system has been thoughtfully designed before a single line of application code is written – exactly as FAANG-level engineering teams operate.

"Build it right. Build it once. Build SYNAPSE."